Data formats

16 December 2025 21:51

JSON

- → Row based
- → Human readable
- → Very inefficient
- → No schema
- → Repeated keys
- → Slow to parse
- → Not good for analytics

✗ Bad for big data

✓ Good for logs, APIs.

Protobuf

- → Binary row based format
- → compact
- → Requires schema
- → Great for messaging, RPC, streaming
- → Not optimized for columnar analytics
- → No column pruning
- → No predicate pushdown
- → No vectorized reads

✓ Great for communication

✗ Bad for analytics engines

Parquet

→ bad at row by row updates + streaming writes

- → Columnar layout
  - data is stored column by column

company_id: [1, 2, 3, 4, ...]

gbg_estimate: [100, 200, 500, 20, ...]

- → Very compressible
- → Easy to skip whole row groups (predicate pushdown)

e.g.: gbg_estimate < 0 → skip block.

- → Column pruning
  - don't read columns you're not using
- → Vectorized scanning
  - batch reads, SIMD operations

So... instead of

row A → all columns
row B → all columns

parquet is... col A= all values, col B= all values

✓ Good: for universal, engine agnostic columnar format.

- Supported by Spark, Trino, Presto, BigQuery, Snowflake, etc.
- ✱ Defacto industry standard.
- ✱ Excellent compression
- ✱ Excellent compression

ORC (optimized row columnar)

- → also columnar format like parquet, but Hive/Hadoop centric.
- You can still use parquet wit hadoop + hive btw.
- often slightly better compression than parquet, but less universal.
- designed for query engines (asking questions about data) rather than inter changes (moving data between systems).

reading as little data as possible, from large datasets.

Query engines: Spark, Hive, Trino, etc.

✱ optimized for columns, statistics (max/min), etc.

- Scan fast, skip aggressively

moving data between services

optimizes for safety rather than speed (e.g.: stable schema, backward/forward compatibility, clear contracts, language neutrality).

✱ Are row based

eg: JSON, protobuf, AVRO, etc.

JSON → human readable; protobuf, AVRO → binary

often one by one / you might replay them later

- write once, read anywhere

Thrift

- → like protobuf, but by meta.
- Declining industry trend btw.
- Focus on serialization & RPC

YAML

- More verbose than JSON, but also human readable.
- Mostly used for configs rather than data

• You can recieve protobuf & convert it to JSON:

✱ GZIP + ZSTD → lower CPU usage, + faster + better compression ratio

→ Compression algorithms

→ they take bytes → make them smaller → restore them later

can be any data (JSON, protobuf, images, etc.)

→ so can do parquet → zstd → parquet.zstd (still parquet).

but parquet also has its own compression.